canaries); how many customers are impacted; and where they are impacted (if we're talking about a global system). Depending on the outcome, you might see an update on a more personalized level, such as the AWS Personal Health Dashboard (http://mng.bz/nW75), or a global view, relevant for all users, as with the GitHub Status (https://www.githubstatus.com/) page.

Regarding the *what* and *how*, the most important thing to remember is to stop the bleeding— that is, try to fix the problem and, ideally, gather data points along the way. The analysis (why it went wrong) comes later; don't let analysis slow down your efforts to stop the bleeding.

To support the person on call, the organization usually develops playbooks and runbooks (http://mng.bz/vn7p) that describe what to check during an incident and documents dependencies to understand downstream effects or possible sources of upstream trouble. Also, some teams practice the processes or material available (including coverage of runbooks) in something usually called a *game day* (http://mng.bz/4DNR).

7.1.3 Learning from the incident after the fact

Let's assume the on-call team did a good job and fixed the problem. That isn't the end of the story. You want to have a process in place that looks at the incident and allows you to achieve two things: understand what the underlying issue was and ensure you don't repeat the same mistake.

I recommend following the path of a blameless postmortem (http://mng.bz/QPO6); in AWS, we call these *corrections of error* (COE; https://wa.aws.amazon.com/wat.concept.coe.en.html). These sessions help you to determine what caused an issue and how to avoid it in the future, typically employing the five whys technique (https://www.lucidchart.com/blog/5-whys-analysis). You may also come across the term *root cause analysis* (RCA; https://www.techtarget.com/searchitoperations/definition/root-cause-analysis), which refers to testing various theories about what originally caused the problem with empirical data.

After this theory-heavy section, let's move on to how to handle notifications for on-call roles—in other words, *alerting*.

7.2 Alerting

In this section, we start by looking at alerting in the Prometheus ecosystem. We then move on to alerting with Grafana, and finally, we discuss alerting offerings of the cloud providers and beyond.

7.2.1 Prometheus alerting

We talked about Prometheus and saw it in action in previous chapters (e.g., in chapters 2 and 5). Now that we have a more complete picture in terms of instrumentation and backends, let's see how Prometheus handles alerting.